

```
$ spring run FunctionalLab --level=0..100
```

Spring Boot × FP

```
// ▯ 0%▯▯▯▯▯▯▯▯ 100%▯▯▯▯▯▯▯▯▯▯
```

0%

▯▯▯

25%

Stream

50%

▯▯▯

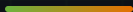
75%

Result

100%

Core/Shell

\$ cat refactor-plan.md



0% / 

NPE 

25% / 

Stream  Optional

50% / 

record  mock

75% → 100% / 

Result  Core/Shell

  bug 

PART 01 — 0%





```
1  @Service
2  public class OrderService {
3      private double total; // 合計金額
4
5      public Order checkout(Long id) {
6          Order order = repo.findById(id).get();
7          total = 0;
8          for (OrderItem item : order.getItems()) {
9              if (item != null
10                 && item.getPrice() != null) {
11                  total += item.getPrice()
12                      * item.getQty();
13              }
14          }
15          if (order.getCoupon() != null) {
16              // ...if ...
17          }
18          order.setTotal(total);
19          emailService.send(order);
20          return repo.save(order);
21      }
22  }
```

SMELL REPORT

- × total 合計 Service 合計
- × .get() NPE
- × null
- ×

0%

NO GLYPH

NO GLYPH

NO GLYPH

NO GLYPH

NO GLYPH

NPE □□□□

findById().get()□getPrice() NO GLYPH NO GLYPH NO GLYPH null

□□□□□□

NO GLYPH NO GLYPH NO GLYPH total NO GLYPH Service □□□□

□□□□□□

NO GLYPH NO GLYPH NO GLYPH NO GLYPH mock repository NO GLYPH email

NO GLYPH A NO GLYPH B

□□□□□□I/O □□□□□

FP □□□□□□NO GLYPHNO GLYPH□□□□□□ — □□□□□□

PART 02 — 25%

Stream Optional

简简单单 JDK 入门



```
1 double total = order.getItems().stream()
2   .filter(Objects::nonNull)
3   .mapToDouble(i -> i.getPrice() * i.getQty())
4   .sum();
```

□□□
□□□□□□□□□□□□□□□□

□□□
□□□□□□□□□□□□□□□□

null → Optional

```
1 Order order = orderRepository.findById(id)
2   .orElseThrow(() -> new OrderNotFoundException(id));
3
4 double discounted = Optional.ofNullable(order.getCoupon())
5   .filter(Coupon::isValid)
6   .map(c -> total * 0.9)
7   .orElse(total);
```

 .get()   null    

order      I/O  

map / filter  FP  Functor       

PART 03 — 50%



□□□□□□□□□□

```
1 public record Pricing(BigDecimal subtotal, BigDecimal total) {}
2
3 public final class PricingRules { // □□□
4     static Pricing price(
5         List<OrderItem> items,
6         Optional<Coupon> coupon) {
7
8         BigDecimal subtotal = /* stream */;
9         BigDecimal total = coupon
10             .filter(Coupon::isValid)
11             .map(c -> subtotal.multiply(RATE_90))
12             .orElse(subtotal);
13         return new Pricing(subtotal, total);
14     }
15 }
```

Service □□

```
1 public Order checkout(Long id) {
2     Order order = findOrder(id);
3     Pricing pricing = PricingRules.price(
4         order.items(), order.coupon());
5     return repo.save(
6         order.withTotal(pricing.total()));
7 }
```


record  setter  DB      



0%



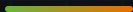
```
1 @SpringBootTest
2 class OrderServiceTest {
3     @MockBean OrderRepository repo;
4     @MockBean EmailService email;
5     // ... stub stub ...
6     // MockitoAnnotations
7 }
```

50%



```
1 @Test
2 void test() {
3     var items = List.of(
4         new OrderItem("00",
5             new BigDecimal("1000"), 1));
6
7     Pricing p = PricingRules.price(
8         items, Optional.of(validCoupon()));
9
10    assertThat(p.total())
11        .isEqualToComparingTo("900");
12 }
```

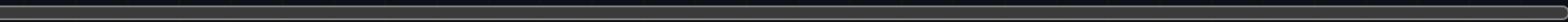
0 mock · 000 · 000



\$ spring run Functionallab



0% ■■■■■



0%

25%

50%

75%

100%

x ■■ null ■■■NPE ■■■■

x setter ■■■■■■■■■■

x ■■■■ I/0■■■■ mock ■■

x ■■■■■■■■■■■■■■

x ■■■■■■■■■■■■■■

PART 04 – 75%

Result



```
1 public sealed interface Result<T> {  
2     record Ok<T>(T value) implements Result<T> {}  
3     record Err<T>(String reason) implements Result<T> {}  
4 }  
5  
6 // 使用 switch 表达式  
7 return switch (result) {  
8     case Ok<Pricing>(Pricing p) -> ResponseEntity.ok(p);  
9     case Err<Pricing>(String reason) ->  
10         ResponseEntity.badRequest().body(reason);  
11 };
```

使用 try-catch 表达式 — 使用 try-catch 表达式

NO GRAPH NO GRAPH NO GRAPH NO GRAPH = NO GRAPH NO GRAPH NO GRAPH NO GRAPH NO GRAPH NO GRAPH

```

1 static UnaryOperator<Pricing> memberDiscount(Member m) {
2     return p -> m.isVip()
3         ? p.withTotal(p.total().multiply(RATE_95))
4         : p;
5 }
6
7 Function<Pricing, Pricing> rules =
8     memberDiscount(member)
9     .andThen(couponDiscount(coupon))
10    .andThen(freeShippingOver(new BigDecimal("1000")));

```

□□□

NO GRAPH NO GRAPH NO GRAPH NO GRAPH NO GRAPH NO GRAPH andThen □□

□□□□□□

NO GRAPH NO GRAPH NO GRAPH NO GRAPH NO GRAPH NO GRAPH FP □

PART 05 — 100%

Functional Core, Imperative Shell

□□□□□□□□□□□□□□



```
1  @Service
2  public class CheckoutService { // Shell
3      public CheckoutResponse checkout(Long id) {
4          Order order = repo.findById(id) // I/O
5              .orElseThrow(...);
6
7          CheckoutDecision decision = CheckoutCore
8              .decide(order.snapshot(), clock.instant());
9
10         return switch (decision) {
11             case Approved a -> {
12                 repo.save(a.pricedOrder());
13                 events.publish(new OrderPriced(a));
14                 yield CheckoutResponse.ok(a);
15             }
16             case Rejected r ->
17                 CheckoutResponse.fail(r.reason());
18         };
19     }
20 }
```



```
1  public final class CheckoutCore {
2      public static CheckoutDecision decide(
3          OrderSnapshot order, Instant now) {
4
5          if (order.items().isEmpty())
6              return new Rejected("???");
7
8          Pricing pricing = /* ??? */;
9          return new Approved(
10              order.withPricing(pricing));
11     }
12 }
```

??????

1. ??????????????????
2. ???????????????????

0% vs 100%

□□	0%	100%
□□□□	mock □□□□□□□□	   mock  □□□
Null □□	NPE □□□	Optional + record □□□□
□□□□	□□□□□□□□	□□□□□□□□□□
□□□□	□□□□□□□□□□	□□□□□□□□□□
□□□□	□□□□□□□□□□	Result + switch □□
□□□□	   if □□	        andThen



Spring

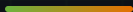
Spring DI FP @Transactional Shell Core

new

JVM TLAB + GC

100%

25% NPE 50% mock



25%

Stream □□□□Optional □□□

50%

record        mock

75%

Result □□□□□andThen □□□

100%

□□□□□□□□□□□□

□□□□□□□□□□□□□□□□□□□□□□

Refactor complete.

```
cd /springboot/fp-in-spring-boot
```

```
$ spring run Functionallab --level=100
```

```
stream → record → result → core/shell
```